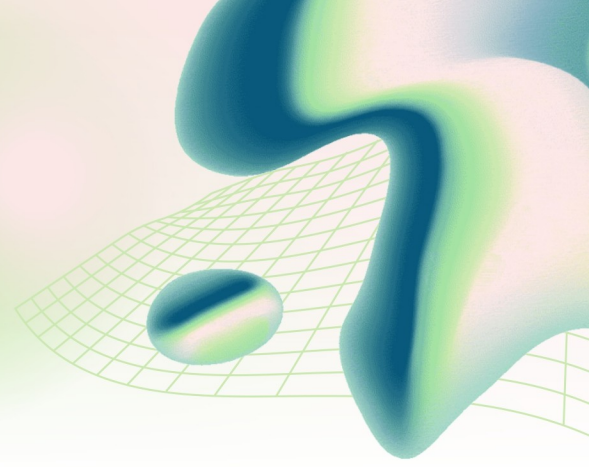


HABITLY

Sistema de Gestión Inmobiliaria en Java



Anexo 03 Gestión de Mantenimiento y Limpieza de Caché

Versión 1.0.1A

Documento de evolución funcional y técnica

Autor: DevNaranjo

Año: 2026

Índice

1. Introducción.....	4
1.1 “Borrar Datos de Aplicación”	4
2. Problema Detectado.....	5
2.1 Consecuencia directa.....	5
3. Objetivo del Anexo.....	6
4. Principio de Diseño Aplicado.....	7
5. ¿Qué significa “Borrar Datos de Aplicación”?.....	8
5.1 Afectaría, principalmente, a elementos como:.....	8
6. Diferencia entre “Datos de Usuario” y “Datos de Sistema”	9
6.1 Datos de usuario.....	9
6.2 Datos de sistema.....	9
7. Justificación Funcional de la Mejora.....	11
7.1 Reinicio controlado del sistema.....	11
7.2 Limpieza de residuos técnicos.....	11
7.3 Mejora de mantenibilidad y soporte.....	12
8. Escenario de Uso Esperado.....	13
8.1 Caso típico.....	13
9. Ubicación Recomendada de la Funcionalidad.....	14
9.1 Ubicación recomendada.....	14
10. Flujo de Confirmación Recomendado.....	15
11. Flujo Funcional Propuesto.....	16
11.1 Paso 1.....	16
11.2 Paso 2.....	16
11.3 Paso 3.....	16
11.4 Paso 4.....	16
11.5 Paso 5.....	16
11.6 Paso 6.....	16
11.7 Paso 7.....	16
11.8 Paso 8.....	16
12. Ejemplo de Comportamiento Esperado en UX.....	17
12.1 Primer mensaje.....	17
12.2 Segunda validación.....	17
12.3 Mensaje final.....	17
12.4 Importancia de esto.....	17
13. Qué debería eliminar esta funcionalidad.....	18
14. Qué NO debería hacer esta funcionalidad.....	19
14.1 Principio clave.....	19
15. Relación con el Protocolo de Rutas (Anexo 01).....	20
15.1 Razón.....	20
15.2 Conclusión.....	20
16. Relación con la Privacidad y Seguridad (Anexo 02).....	21
17. Consideración sobre Crecimiento del Inventario.....	22

Anexo 03 – v1.0.1A

17.1 Implicación técnica.....	22
18. Recomendación de Arquitectura Técnica.....	23
18.1 Posibles nombres recomendados.....	23
19. Flujo Técnico Esperado.....	24
19.1 Fase 1 — Verificación.....	24
19.2 Fase 2 — Confirmación.....	24
19.3 Fase 3 — Ejecución.....	24
19.4 Fase 4 — Reconstrucción controlada.....	24
19.5 Fase 5 — Feedback.....	24
20. Posible Evolución Futura de la Funcionalidad.....	25
20.1 Evoluciones recomendables a medio plazo.....	25
21. Requisitos Funcionales Asociados.....	26
21.1 RF-III.1.....	26
21.2 RF-III.2.....	26
21.3 RF-III.3.....	26
21.4 RF-III.4.....	26
21.5 RF-III.5.....	26
22. Requisitos No Funcionales Asociados.....	27
22.1 RNF-III.1 — Seguridad Operativa.....	27
22.2 RNF-III.2 — Robustez.....	27
22.3 RNF-III.3 — Usabilidad.....	27
22.4 RNF-III.4 — Mantenibilidad.....	27
22.5 RNF-III.5 — Escalabilidad.....	27
23. Riesgos si No se Implementa esta Mejora.....	28
24. Conclusión.....	29

1. INTRODUCCIÓN

A medida que Habitly evoluciona desde una aplicación académica funcional hacia un sistema más sólido y mantenible, no solo debe preocuparse por **guardar datos correctamente** y **protegerlos**, sino también por ofrecer mecanismos de **mantenimiento interno** que permitan conservar el sistema limpio, controlado y sostenible a lo largo del tiempo.

En la versión **v1.0.1-BETA**, el sistema ya dispone de:

- persistencia local mediante .dat,
- lógica financiera funcional,
- control de estados automáticos,
- y validación reforzada de entrada de datos.

Sin embargo, todavía no existe una funcionalidad específica que permita al usuario **eliminar de forma segura los datos locales de la aplicación** cuando así lo desee.

Este anexo define el diseño funcional y técnico de una futura opción de mantenimiento denominada:

1.1 “Borrar Datos de Aplicación”

Su objetivo será permitir la **limpieza controlada del almacenamiento local de Habitly**, evitando acumulación innecesaria de datos, facilitando pruebas, reinicios o mantenimiento, y mejorando la sostenibilidad general del sistema.

2. PROBLEMA DETECTADO

En un sistema con persistencia local, el almacenamiento no desaparece al cerrar el programa. Eso es una ventaja, pero también implica una responsabilidad técnica.

Si Habitly sigue creciendo, el entorno local del usuario puede empezar a acumular:

- inventarios antiguos,
- archivos temporales,
- registros obsoletos,
- futuras cachés,
- backups,
- o configuraciones ya innecesarias.

Actualmente, el sistema no dispone de ninguna herramienta interna para gestionar este escenario.

2.1 Consecuencia directa

Si el usuario desea “resetear” la aplicación, liberar espacio o empezar de cero, la única opción sería **borrar archivos manualmente desde el sistema de carpetas**, lo cual:

- no es intuitivo,
- no es seguro,
- y rompe la experiencia de uso.

Por ello, Habitly necesita incorporar una funcionalidad oficial de limpieza interna.

3. OBJETIVO DEL ANEXO

El objetivo de esta mejora es diseñar una funcionalidad de mantenimiento que permita al usuario:

- borrar de forma intencionada los datos persistentes de la aplicación,
- liberar espacio en almacenamiento local,
- reiniciar el sistema si lo necesita,
- y mantener el entorno interno de Habitly limpio y controlado.

Esta funcionalidad no debe entenderse como una operación casual, sino como una **herramienta de mantenimiento avanzada**, diseñada para actuar con seguridad y confirmación explícita.

4. PRINCIPIO DE DISEÑO APLICADO

La propuesta se apoya en un principio muy importante de diseño de software:

Toda aplicación que genera almacenamiento persistente debería ofrecer mecanismos internos de limpieza o reinicio controlado.

Esto evita que el usuario dependa de acciones externas o manipulaciones manuales sobre el sistema de archivos.

En otras palabras:

- Habitly debe ser capaz de **crear sus propios datos**,
- pero también de **gestionarlos y eliminarlos correctamente** cuando sea necesario.

5. ¿QUÉ SIGNIFICA “BORRAR DATOS DE APLICACIÓN”?

Dentro del contexto de Habitly, esta funcionalidad no se refiere únicamente a “borrar un archivo”.

Se refiere a la eliminación controlada de los elementos locales que la aplicación haya generado y que formen parte de su entorno persistente.

5.1 Afectaría, principalmente, a elementos como:

- archivo de inventario principal
- posibles archivos de caché futura
- backups internos
- logs locales
- configuraciones temporales
- otros residuos operativos de la aplicación

6. DIFERENCIA ENTRE “DATOS DE USUARIO” Y “DATOS DE SISTEMA”

Para diseñar correctamente esta funcionalidad, es importante distinguir dos tipos de información:

6.1 Datos de usuario

Son los datos funcionales que tienen valor directo para el uso de Habitly.

Ejemplos:

- viviendas registradas
- estados de alquiler
- pagos
- historial financiero
- datos de propietarios o arrendatarios

6.2 Datos de sistema

Son datos auxiliares necesarios para el funcionamiento técnico del programa.

Ejemplos:

- caché
- archivos temporales
- logs
- backups
- configuración interna

Importancia de esta distinción

En una primera implementación, la opción **“Borrar Datos de Aplicación”** puede plantearse de forma global. Pero a medio plazo, Habitly podría evolucionar hacia una gestión más fina, por ejemplo:

- borrar solo caché
- borrar solo backups
- reiniciar completamente la aplicación

Esto hace que la arquitectura de limpieza deba pensarse ya con visión escalable.

7. JUSTIFICACIÓN FUNCIONAL DE LA MEJORA

Esta funcionalidad es útil por varios motivos reales.

7.1 Reinicio controlado del sistema

Habrán situaciones en las que el usuario quiera:

- empezar desde cero,
- rehacer el inventario,
- eliminar pruebas anteriores,
- o resetear Habitly tras un uso de prueba.

Sin una función oficial, esto obligaría a intervenir manualmente en carpetas internas.

7.2 Limpieza de residuos técnicos

Con futuras versiones, Habitly puede llegar a generar más contenido local del necesario.

Por ejemplo:

- caché temporal,
- archivos intermedios,
- backups duplicados,
- logs acumulados.

Si no existe una estrategia de limpieza, el almacenamiento puede crecer sin control.

7.3 Mejora de mantenibilidad y soporte

Una función de limpieza también resulta útil desde el punto de vista técnico, porque:

- facilita pruebas,
- simplifica soporte,
- reduce incidencias por datos corruptos o antiguos,
- y permite restaurar el sistema a un estado limpio sin reinstalaciones manuales.

8. ESCENARIO DE USO ESPERADO

El uso esperado de esta funcionalidad sería el siguiente:

8.1 Caso típico

Un usuario ha trabajado durante un tiempo con Habitly, ha hecho pruebas o ha generado inventarios que ya no necesita.

Desea:

- liberar espacio,
- reiniciar la aplicación,
- o limpiar el sistema antes de comenzar una nueva fase de uso.

En ese momento, Habitly debería ofrecer una opción específica para ello.

9. UBICACIÓN RECOMENDADA DE LA FUNCIONALIDAD

Desde el punto de vista de experiencia de usuario, esta opción **no debería colocarse entre las acciones normales de negocio** como:

- registrar vivienda,
- registrar pago,
- listar inventario,
- etc.

Eso sería peligroso.

9.1 Ubicación recomendada

Debe ubicarse dentro de un apartado específico como:

Mantenimiento

o

Configuración avanzada

Esto reduce el riesgo de uso accidental y refuerza el carácter técnico de la operación.

10. FLUJO DE CONFIRMACIÓN RECOMENDADO

Debido a que esta operación puede implicar pérdida irreversible de datos locales, el sistema **no debe ejecutarla con una sola acción directa**.

Debe existir un **flujo de confirmación reforzado**.

11. FLUJO FUNCIONAL PROPUESTO

El comportamiento recomendado para la opción “**Borrar Datos de Aplicación**” debe ser el siguiente:

11.1 Paso 1

El usuario accede al menú de mantenimiento o configuración avanzada.

11.2 Paso 2

Selecciona la opción:

Borrar Datos de Aplicación

11.3 Paso 3

Habitly muestra una advertencia clara explicando que esta operación eliminará los datos locales persistentes.

11.4 Paso 4

El sistema solicita una primera confirmación explícita.

11.5 Paso 5

Si el usuario confirma, el sistema muestra una segunda validación de seguridad.

11.6 Paso 6

Solo tras la segunda confirmación se ejecuta el proceso de limpieza.

11.7 Paso 7

El programa informa al usuario del resultado de la operación.

11.8 Paso 8

Habitly queda preparado para reiniciarse con almacenamiento limpio o reconstruir su estructura base automáticamente.

12. EJEMPLO DE COMPORTAMIENTO ESPERADO EN UX

Aunque el diseño final podrá variar, la filosofía de interacción debería seguir este patrón:

12.1 Primer mensaje

- advertencia clara
- lenguaje directo
- explicación de consecuencias

12.2 Segunda validación

- confirmación reforzada
- evita pulsaciones o elecciones impulsivas

12.3 Mensaje final

- resultado confirmado
- feedback claro de éxito o fallo

12.4 Importancia de esto

La experiencia debe dejar claro al usuario que:

no está “limpiando una pantalla”, sino **borrando datos reales almacenados por la aplicación.**

13. QUÉ DEBERÍA ELIMINAR ESTA FUNCIONALIDAD

En una primera implementación razonable, Habitly debería poder eliminar al menos:

- archivo principal de persistencia
- futuros archivos temporales
- posibles cachés
- estructuras de apoyo internas

Elemento mínimo prioritario

El primer objetivo práctico debe ser el archivo:

`inventario_habitly.dat`

ya que es el núcleo de persistencia actual.

14. QUÉ NO DEBERÍA HACER ESTA FUNCIONALIDAD

Para evitar comportamientos peligrosos o poco mantenibles, esta función no debería:

- borrar archivos fuera del entorno de Habitly,
- eliminar carpetas del proyecto fuente,
- afectar al código Java,
- modificar el repositorio Git,
- tocar documentación o archivos del desarrollador,
- actuar fuera del directorio interno de datos de aplicación.

14.1 Principio clave

La limpieza debe limitarse estrictamente al entorno de almacenamiento controlado por Habitly.

Esto es especialmente importante si el sistema empieza a usar rutas del tipo:

- %AppData%\Habitly
- ~/.habitly

como se describe en el **Anexo 01**.

15. RELACIÓN CON EL PROTOCOLO DE RUTAS (ANEXO 01)

Esta funcionalidad depende directamente del sistema de almacenamiento definido en el Anexo I.

15.1 Razón

Si Habitty guarda sus datos en una carpeta de aplicación estructurada, entonces puede también:

- identificar claramente qué pertenece a la app,
- limpiar sus propios residuos,
- y gestionar mejor su ciclo de vida local.

15.2 Conclusión

No tiene sentido diseñar una limpieza robusta si antes no existe una estructura ordenada de almacenamiento.

Por eso ambos anexos están íntimamente conectados.

16. RELACIÓN CON LA PRIVACIDAD Y SEGURIDAD (ANEXO 02)

La funcionalidad de borrado también tiene una dimensión de privacidad.

Eliminar correctamente los datos locales puede ser importante en escenarios como:

- cambio de usuario del equipo,
- cierre de uso de la aplicación,
- limpieza de pruebas con datos sensibles,
- mantenimiento previo a migraciones,
- o simplemente control del almacenamiento local.

Por tanto, esta mejora no solo afecta al mantenimiento técnico, sino también a la **higiene de datos**.

17. CONSIDERACIÓN SOBRE CRECIMIENTO DEL INVENTARIO

Aunque en la fase actual Habitly aún no maneje volúmenes masivos de datos, es importante prever el comportamiento del sistema a medida que el inventario crezca.

Con el tiempo, podrían acumularse:

- más viviendas,
- más estados,
- más pagos,
- más historiales,
- más backups,
- y más metadatos asociados.

Esto puede hacer que el almacenamiento local aumente progresivamente.

17.1 Implicación técnica

Una funcionalidad de limpieza evita que Habitly se convierta en una aplicación que:

- solo sabe guardar,
- pero no sabe mantener su propio ecosistema de datos.

18. RECOMENDACIÓN DE ARQUITECTURA TÉCNICA

Desde el punto de vista de diseño, esta funcionalidad no debería implementarse de forma improvisada dentro del main.

Lo recomendable es que exista una capa o clase responsable de las tareas de mantenimiento interno.

18.1 Posibles nombres recomendados

GestorMantenimiento

o

AdministradorSistemaLocal

Responsabilidades sugeridas

- localizar el directorio de datos de Habitly
- comprobar qué archivos existen
- borrar archivos permitidos
- confirmar estados de limpieza
- devolver mensajes de resultado

Esto evita mezclar lógica de sistema con lógica de negocio.

19. FLUJO TÉCNICO ESPERADO

A nivel interno, la funcionalidad debería seguir esta lógica general:

19.1 Fase 1 – Verificación

- localizar la carpeta de datos
- comprobar si existen archivos o subdirectorios relevantes

19.2 Fase 2 – Confirmación

- bloquear la acción hasta obtener confirmación explícita del usuario

19.3 Fase 3 – Ejecución

- borrar solo los archivos autorizados

19.4 Fase 4 – Reconstrucción controlada

- si la app necesita estructura mínima, podrá regenerarla en el siguiente inicio

19.5 Fase 5 – Feedback

- informar claramente si la limpieza fue correcta o si hubo errores

20. POSIBLE EVOLUCIÓN FUTURA DE LA FUNCIONALIDAD

Aunque la primera versión de esta herramienta puede ser simple, su diseño debería permitir crecer en el futuro.

20.1 Evoluciones recomendables a medio plazo

- borrar solo caché
- borrar solo backups
- borrar solo logs
- restaurar configuración por defecto
- reinicio completo de Habitly
- limpieza con exportación previa opcional

Esto permitiría pasar de una limpieza “todo o nada” a una administración más precisa.

21. REQUISITOS FUNCIONALES ASOCIADOS

Para considerar correctamente implementada esta mejora, Habitly deberá cumplir los siguientes requisitos:

21.1 RF-III.1

El sistema deberá ofrecer una opción explícita para borrar los datos locales de aplicación.

21.2 RF-III.2

La operación deberá requerir confirmación reforzada por parte del usuario.

21.3 RF-III.3

El sistema deberá eliminar únicamente los archivos y estructuras controladas por Habitly.

21.4 RF-III.4

La aplicación deberá informar claramente del resultado de la operación.

21.5 RF-III.5

Tras la limpieza, Habitly deberá poder seguir funcionando correctamente y reconstruir su estructura básica si es necesario.

22. REQUISITOS NO FUNCIONALES ASOCIADOS

22.1 RNF-III.1 — Seguridad Operativa

La limpieza no deberá afectar a archivos ajenos al entorno de Habitly.

22.2 RNF-III.2 — Robustez

La operación deberá tolerar que algunos archivos no existan o ya hayan sido eliminados.

22.3 RNF-III.3 — Usabilidad

El usuario deberá comprender claramente las consecuencias de la acción.

22.4 RNF-III.4 — Mantenibilidad

La lógica de limpieza deberá estar desacoplada del flujo principal de negocio.

22.5 RNF-III.5 — Escalabilidad

La solución deberá permitir ampliar el mantenimiento a otros tipos de datos en el futuro.

23. RIESGOS SI NO SE IMPLEMENTA ESTA MEJORA

Si Habitty sigue creciendo sin una funcionalidad interna de mantenimiento, pueden aparecer varios problemas:

- acumulación innecesaria de archivos locales,
- dificultad para reiniciar el sistema de forma limpia,
- dependencia del borrado manual,
- riesgo de errores por datos antiguos o corruptos,
- peor experiencia de soporte y pruebas,
- y falta de control sobre el ciclo de vida del almacenamiento.

A largo plazo, esto se traduce en **desorden técnico y menor profesionalidad del sistema.**

24. CONCLUSIÓN

La funcionalidad “**Borrar Datos de Aplicación**” constituye una mejora de mantenimiento altamente recomendable para la evolución de Habitly.

No se trata simplemente de añadir una opción de borrado, sino de incorporar una **política de limpieza controlada, segura y coherente con el crecimiento del proyecto**.

Su implementación permitirá a Habitly:

- gestionar mejor su almacenamiento local,
- facilitar reinicios y pruebas,
- mejorar la higiene del sistema,
- reforzar la privacidad operativa,
- y preparar el terreno para versiones más complejas.

En combinación con el **Anexo 01 (Rutas de Sistema)** y el **Anexo 02 (Cifrado AES)**, este anexo completa una base técnica muy sólida para la maduración de la persistencia local de Habitly de cara a la futura línea **v1.1**.